

SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSECODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 3

Duration:1 Hour

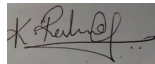
Total Marks:50

Annexure A
Scenario Based Assignment

Code of Conduct:

I K. Reshma (192565062) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A

Scenario Based Assignment

Student 22: Kadapa Reshma | Register Number: 192565062

1. Scenario

A large agricultural enterprise implements an AI system to optimize water distribution through a network of smart irrigation valves across thousands of acres. The objective is to minimize water wastage, energy consumption, and crop stress. The system must continuously adjust valve settings based on soil-moisture sensor readings. However, the search space is extremely large, weather patterns change dynamically, and the system may get stuck in locally optimal valve configurations.

Tasks:

Formulate this as an optimization problem and explain why traditional search is inefficient.

- i. Explain how Hill Climbing would work in this scenario and identify its limitations.
- ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation.
- iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations.
- iv. Recommend the best approach and justify your choice based on scalability and adaptability.

2. Scenario

An autonomous submarine is deployed to explore an uncharted ocean trench. It must: navigate safely to map the trench floor, avoid hazards (rock formations, thermal vents), operate with limited sonar range, and make decisions without a complete map. The submarine updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

- i. Explain why this problem requires an online search agent instead of offline search.
- ii. Analyze the challenges of partial observability and dynamic environments.
- iii. Describe how the submarine builds and updates its internal model.
- iv. Compare online search with uninformed and informed search strategies.
- v. Justify the most suitable approach considering uncertainty, adaptability, and safety.

3. Scenario

A hospital is designing an AI system to automatically generate monthly duty rosters for its nursing staff. The system must satisfy multiple constraints: no nurse should be scheduled for two overlapping shifts, a limited number of nurses are qualified for the ICU, certain senior nurses must supervise night shifts, nurses have individual leave and availability constraints,

and special accommodations are needed for part-time staff. The complexity increases as the number of nurses and wards grows.

Tasks:

Formulate the problem as a Constraint Satisfaction Problem (CSP).

- i. Clearly define variables, domains, and constraints with explanation.

ANSWER:

1. AI-Based Smart Irrigation Water Distribution

Scenario

A large agricultural enterprise uses an AI system to optimize water distribution through smart irrigation valves across thousands of acres. The main objectives are to minimize water wastage, energy consumption, and crop stress. The system continuously adjusts valve settings using soil-moisture sensor readings. Since the search space is very large and weather conditions change dynamically, the system may become trapped in locally optimal configurations.

Formulation as an Optimization Problem

The irrigation system can be formulated as an optimization problem where the state represents the current settings of all irrigation valves and the solution represents the best combination of valve settings.

Let:

- Variables: Valve settings for each irrigation zone.
- Input: Soil-moisture readings, weather conditions, crop requirements, and water availability.
- Actions: Open, close, or adjust the flow rate of valves.
- Objective: Minimize water wastage, energy consumption, and crop stress.
- Constraints: Available water, valve capacity, soil conditions, crop requirements, and operating time.

A suitable objective function can be represented as:

Minimize:

Cost = Water Wastage + Energy Consumption + Crop Stress

The AI system searches for a valve configuration that gives the lowest overall cost.

Why Traditional Search is Inefficient

Traditional exhaustive search is inefficient because thousands of acres may contain a very large number of valves. If each valve has several possible settings, the number of possible combinations increases exponentially.

For example, if 100 valves have 5 possible settings each:

Number of configurations = 5^{100}

This creates an extremely large search space. Checking every possible configuration would require enormous computational time. In addition, weather and soil conditions change continuously, making previously calculated solutions less useful.

Therefore, heuristic optimization methods such as Hill Climbing, Simulated Annealing, or Genetic Algorithms are more suitable.

i. Hill Climbing in the Smart Irrigation Scenario

Hill Climbing is a local search algorithm that begins with an initial solution and repeatedly moves to a neighboring solution that improves the objective value.

Working

1. Start with an initial valve configuration.
2. Measure soil moisture and calculate the current cost.
3. Generate neighboring valve configurations.
4. Compare the neighboring configurations.
5. Select a better configuration.
6. Adjust the irrigation valves.
7. Repeat the process until no better neighboring solution is found.

For example, if one irrigation zone is receiving too much water, the algorithm can reduce its valve opening. If this reduces water wastage without increasing crop stress, the new configuration becomes the current solution.

Limitations of Hill Climbing

- It considers mainly local improvements.
 - It can become trapped in a local optimum.
 - It may have difficulty moving through flat regions.
 - It can struggle with complex search spaces.
 - A poor initial configuration can lead to a poor final solution.
 - Dynamic weather conditions may make the current optimum change frequently.
-

ii. Local Maxima, Plateaus and Ridges

Local Maximum

A local maximum is a solution that is better than its immediate neighboring solutions but is not the best solution overall.

Real-world interpretation:

The irrigation system may find a valve configuration that significantly reduces water consumption in one region. However, another configuration involving several distant valves may provide much better overall water efficiency. Hill Climbing may stop before reaching this better configuration.

Plateau

A plateau is a region where many neighboring solutions have approximately the same evaluation value.

Real-world interpretation:

Changing one valve slightly may produce almost no change in water wastage or crop stress. Therefore, the algorithm cannot identify a clear direction for improvement.

Ridge

A ridge is a narrow region where improvement requires a sequence of coordinated changes rather than a simple change to one variable.

Real-world interpretation:

Changing the settings of several valves together may be necessary to improve irrigation efficiency. Changing only one valve may not produce an improvement, causing Hill Climbing to miss the better solution.

iii. Simulated Annealing and Genetic Algorithms

Simulated Annealing

Simulated Annealing improves upon Hill Climbing by sometimes accepting a worse solution.

At high temperature, the algorithm has a greater probability of accepting a poorer solution. This allows it to escape local optima. As the temperature gradually decreases, the algorithm becomes more selective and focuses on improving the solution.

Advantages:

- Escapes local maxima.
- Can explore different regions of the search space.
- Suitable for large optimization problems.
- Provides a balance between exploration and exploitation.

Genetic Algorithm

A Genetic Algorithm uses concepts inspired by natural evolution.

The main steps are:

1. Generate an initial population of irrigation solutions.
2. Evaluate the fitness of each solution.
3. Select better solutions.
4. Perform crossover to create new solutions.
5. Apply mutation to introduce variation.
6. Evaluate the new population.

7. Repeat until a satisfactory solution is obtained.

For example, one irrigation configuration may provide good water efficiency while another provides low energy consumption. Crossover can combine useful characteristics from both configurations.

iv. Recommended Approach

For this scenario, Genetic Algorithm is a strong choice because the irrigation system has a very large search space involving thousands of valves.

It can maintain multiple possible solutions simultaneously and explore different regions of the search space. Mutation allows new valve configurations to be generated, while crossover combines useful characteristics.

However, because weather and soil conditions change dynamically, the Genetic Algorithm should be periodically updated using new sensor information.

Justification

Requirement	Genetic Algorithm
Large search space	Suitable
Avoiding local optima	Good
Multiple possible solutions	Maintains population
Adaptability	High
Scalability	Good
Dynamic conditions	Can be repeatedly updated

Therefore, a Genetic Algorithm with continuous sensor-based updates is recommended for large-scale smart irrigation optimization.

2. Autonomous Submarine Exploration

Scenario

An autonomous submarine explores an uncharted ocean trench. It must navigate safely, map the trench, avoid hazards such as rock formations and thermal vents, and operate with limited sonar range. Since the complete map is unavailable, the submarine continuously updates its knowledge while exploring.

i. Need for an Online Search Agent

This problem requires an online search agent because the submarine does not have complete information about the environment before starting its journey.

An offline search algorithm assumes that the complete problem description, including the state space and possible paths, is available before the search begins.

In this scenario:

- The ocean trench is uncharted.
- The complete map is unavailable.
- Sonar has limited range.
- New obstacles may be discovered during navigation.
- The submarine must make decisions while moving.
- Information is obtained through sensors.

Therefore, the submarine must act, observe, update its knowledge, and search again.

This is the main reason an online search agent is more appropriate than an offline search method.

ii. Partial Observability and Dynamic Environment

Partial Observability

The submarine cannot observe the entire environment at once because its sonar has limited range.

For example, a rock formation may be outside the current sonar range. The submarine becomes aware of it only when it moves closer.

This creates uncertainty about:

- Location of obstacles.
- Depth of the ocean floor.
- Safe navigation routes.
- Location of thermal vents.
- Distance to unexplored regions.

Dynamic Environment

The environment can also change while the submarine is operating.

Examples include:

- Changes in underwater currents.
- Movement of marine objects.
- Changes around thermal vents.
- Sensor noise.

- Changes in navigation conditions.

Therefore, a route that was safe earlier may become less suitable later.

iii. Building and Updating the Internal Model

The submarine builds an internal model using sensor information.

The process can be represented as:

Sense → Update Map → Evaluate Situation → Select Action → Move → Sense Again

Step 1: Sense

The submarine collects information using sonar and other sensors.

Step 2: Update Map

New information about obstacles, depth, and unexplored areas is stored in an internal map.

Step 3: Identify Safe and Unsafe Areas

The system marks discovered hazards and identifies possible safe routes.

Step 4: Select an Action

The submarine chooses whether to move forward, change direction, slow down, or explore another route.

Step 5: Repeat

After taking an action, it obtains new information and updates the internal model again.

This continuous process allows the submarine to gradually build a better understanding of the unknown environment.

iv. Online Search vs Uninformed and Informed Search

Feature	Uninformed Search	Informed Search	Online Search
Complete map required	Usually yes	Usually yes	No
Uses heuristic	No	Yes	Can use available information
Works with unknown environment	Poorly	Limited	Very suitable
Learns while moving	No	Generally no	Yes
Handles new information	Limited	Limited	Strong
Suitable for submarine	Low	Moderate	High

Uninformed Search

Methods such as Breadth-First Search and Depth-First Search do not use additional knowledge about the quality of states.

Informed Search

Methods such as A* use heuristic information to guide the search toward a goal.

Online Search

Online search allows the agent to discover the environment while performing actions. This is particularly useful when the environment is unknown.

v. Most Suitable Approach

The online search approach is most suitable for the autonomous submarine.

It provides:

- Adaptability to newly discovered information.
- Ability to operate without a complete map.
- Continuous environmental learning.
- Better handling of uncertainty.
- Safer decision-making.

For safety-critical navigation, the online search agent should also prioritize hazard avoidance over simply reaching the destination quickly.

Therefore, an online, sensor-driven search strategy is recommended because it provides the necessary uncertainty handling, adaptability, and safety.

3. Hospital Nursing Duty Roster as a Constraint Satisfaction Problem

Scenario

A hospital wants to automatically generate monthly duty rosters for nursing staff. The system must ensure that nurses are not assigned overlapping shifts, only qualified nurses are assigned to ICU duties, senior nurses supervise night shifts, individual leave and availability are respected, and part-time nurses receive suitable schedules.

This problem can be formulated as a Constraint Satisfaction Problem (CSP).

A CSP consists of:

1. Variables
2. Domains
3. Constraints

The objective is to assign values to all variables while satisfying all required constraints.

i. Variables, Domains and Constraints

A. Variables

Variables represent the decisions that must be made by the scheduling system.

For example:

- N1_D1 = Nurse 1's duty on Day 1.
- N1_D2 = Nurse 1's duty on Day 2.
- N2_D1 = Nurse 2's duty on Day 1.
- N2_D2 = Nurse 2's duty on Day 2.

More generally:

Variable = Nurse + Date + Shift/Ward

For example:

X(nurse, date, shift)

The system must determine which nurse is assigned to which shift and ward.

B. Domains

A domain represents the possible values that can be assigned to a variable.

Possible shift values include:

- Morning
- Afternoon
- Night
- Off/Leave

Possible ward values include:

- General Ward
- ICU
- Emergency
- Pediatric Ward
- Other hospital departments

For example:

Domain(Nurse 1, Day 1):

{Morning, Afternoon, Night, Off}

However, the actual domain is reduced according to the nurse's availability and qualifications.

For an ICU-qualified nurse:

{General Morning, General Afternoon, ICU Morning, ICU Afternoon, ICU Night, Off}

For a nurse who is not ICU-qualified, ICU assignments are removed from the domain.

C. Constraints

Constraints are rules that must be satisfied by the final roster.

Constraint 1: No Overlapping Shifts

A nurse cannot be assigned to two shifts occurring at the same time.

For example:

Nurse A $\neq$ Morning Shift and Afternoon Shift at the same time.

This prevents duplicate scheduling.

Constraint 2: ICU Qualification

Only nurses who are qualified to work in the ICU can be assigned to ICU shifts.

For example:

ICU Shift $\rightarrow$ Nurse must be ICU Qualified

If Nurse A is not ICU-qualified, the system cannot assign Nurse A to an ICU duty.

Constraint 3: Senior Nurse Supervision

Certain senior nurses must supervise night shifts.

Therefore:

Night Shift $\rightarrow$ At least one required senior nurse

This ensures appropriate supervision during night operations.

Constraint 4: Leave Constraint

A nurse who is on approved leave cannot be assigned a duty.

For example:

If Nurse A has leave on Day 10:

Nurse A, Day 10 = Leave

Therefore, Nurse A cannot be assigned to Morning, Afternoon, or Night duty on that day.

Constraint 5: Availability Constraint

The system must consider individual nurse availability.

For example, if a nurse is unavailable on Sundays, the system must not assign that nurse to Sunday shifts.

Constraint 6: Part-Time Nurse Constraint

Part-time nurses should only be assigned according to their working hours and availability.

For example:

Part-time Nurse → Maximum allowed shifts per week

This prevents assigning excessive duties to part-time staff.

Constraint 7: Shift Coverage

Every required shift must have an appropriate number of nurses.

For example:

Morning Shift → Minimum 5 nurses

Night Shift → Minimum 3 nurses

The exact number can be determined by hospital requirements.

Constraint 8: Workload Balance

The system should distribute duties fairly among nurses.

For example, one nurse should not receive significantly more night shifts than another nurse with similar availability and qualifications.

CSP Representation

The complete CSP can be represented as:

CSP = (Variables, Domains, Constraints)

Variables

$X = \{\text{Nurse-Day-Shift assignments}\}$

Domains

$D = \{\text{Morning, Afternoon, Night, Off/Leave}\}$

with ward assignments such as:

$\{\text{General Ward, ICU, Emergency, Pediatric Ward}\}$

Constraints

$C = \{\text{No Overlap, ICU Qualification, Senior Supervision, Leave, Availability, Part-Time Rules, Shift Coverage, Workload Balance}\}$

The goal is to find an assignment where:

All variables receive valid values and all constraints are satisfied.

Example

Suppose there are three nurses:

- Nurse A – Senior and ICU qualified
- Nurse B – Junior and ICU qualified
- Nurse C – Junior and not ICU qualified

For a night ICU shift:

- Nurse A $\rightarrow$ Allowed
- Nurse B $\rightarrow$ Allowed
- Nurse C $\rightarrow$ Not Allowed

If Nurse A is on leave, the system must choose Nurse B.

This demonstrates how constraints reduce the available domain and help the AI system generate a valid roster.

Backtracking Search

Backtracking is a systematic CSP-solving technique.

The algorithm assigns values to variables one at a time.

Working

1. Select an unassigned variable.
2. Select a value from its domain.
3. Check whether the assignment satisfies constraints.
4. If valid, continue with the next variable.
5. If invalid, select another value.
6. If no value works, go back to the previous variable.

7. Change the previous assignment.
8. Continue until a complete valid roster is obtained.

Example

Suppose the system assigns:

Nurse A → Monday Night

Then it checks whether this violates:

- Leave constraint.
- Availability constraint.
- Senior supervision constraint.
- Maximum working hours.
- Other shift constraints.

If a violation occurs, the system backtracks and tries another assignment.

Constraint Propagation and Forward Checking

Constraint propagation improves the efficiency of CSP solving.

Forward checking removes values from the domains of future variables that would violate the constraints.

For example, if Nurse A is assigned to Monday Morning, the system can remove overlapping Monday Morning assignments for Nurse A from future possibilities.

Similarly, if Nurse C is not ICU-qualified, ICU assignments are removed from Nurse C's domain immediately.

Benefits

- Reduces the search space.
 - Detects conflicts earlier.
 - Reduces unnecessary backtracking.
 - Speeds up roster generation.
 - Improves scalability for large hospitals.
-

Real-World Challenges

The hospital scheduling problem becomes difficult when the number of nurses, wards, shifts, and constraints increases.

Important challenges include:

1. Large number of nurses.
2. Multiple hospital departments.
3. Different qualifications.
4. Individual leave requests.
5. Part-time staff.
6. Night-shift requirements.
7. ICU staffing requirements.
8. Fair workload distribution.
9. Unexpected staff absence.
10. Changes in hospital workload.

A small change, such as one nurse taking emergency leave, may require several assignments to be changed.

Best Strategy for Scalability

For a large hospital, Backtracking Search combined with constraint propagation/forward checking is a suitable strategy.

Backtracking provides systematic exploration of possible rosters, while constraint propagation removes invalid choices before they are explored.

The approach can be further improved by using heuristics such as:

- Minimum Remaining Values (MRV): Select the variable with the smallest remaining domain.
- Degree Heuristic: Select the variable involved in the largest number of constraints.
- Least Constraining Value: Choose the value that leaves the maximum flexibility for other variables.

Final Recommendation

A CSP solver using backtracking with forward checking and heuristic variable selection is recommended.

It can efficiently handle:

- Nurse availability.
- ICU qualification.
- Night supervision.
- Leave constraints.
- Part-time requirements.

- Shift overlap.
 - Increasing numbers of nurses and wards.
-

Overall Conclusion

The three scenarios demonstrate different AI problem-solving techniques.

Scenario 1 – Smart Irrigation:

The problem is a large optimization problem. Hill Climbing is simple but can become trapped in local maxima, plateaus, and ridges. A Genetic Algorithm is more suitable because it can explore multiple solutions and handle a large search space.

Scenario 2 – Autonomous Submarine:

The submarine operates in an unknown environment with limited information. Therefore, an online search agent is appropriate because it can continuously sense, learn, update its internal model, and make decisions while exploring.

Scenario 3 – Hospital Duty Roster:

The scheduling problem can be represented as a CSP using variables, domains, and constraints. Backtracking combined with forward checking and heuristics provides an effective and scalable solution.

Thus, the scenarios show how optimization algorithms, online search, and CSP techniques can be selected according to the characteristics of real-world AI problems.

- ii. Explain how backtracking search works in this scenario.
- iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency.
- iv. Analyze real-world challenges and justify the best strategy for scalability.